动态规划

A 通天之分组背包 洛谷 P1757

思路

背包模板题

代码

```
#include<bits/stdc++.h>
#define int long long//注意要开long long
using namespace std;
typedef long long ll;
typedef long double ld;
const int maxn=1010;
void solve()
{
    int m, n;
    cin >> m >> n;
    vector<vector<int>> w(n + 1);
    vector<vector<int>> v(n + 1);
    int maxk = -1;//最大组数
    for (int i = 1;i <= n;i++)
    {
        long long a, b, c;
        cin >> a >> b >> c;
        maxk = max(c, maxk);
        w[c].push_back(a);
        v[c].push_back(b);
    }
    vector<int> dp(m+1,0);//表示重量为i时 最大的利用价值 这里滚动压缩了一维

    //注意：初学的背包模板一般是dp[i][j] 表示考虑到第i组 重量为j的最优利用价值
    //转移方程为dp[i][j]=max(dp[i][j],dp[i-1][j-w]+v)
    //所以考虑到第i组重量为j时 状态依赖于第i-1组重量为(j-w)的状态
    //所以压缩维度后要倒序更新 以免i-1行的j-w被第i行的覆盖

    for (int j =1;j<=maxk;j++)
    {
        for (int i = m;i >= 1;i--)//倒序更新
        {
            for (int k = 0;k < w[j].size();k++)
            {
                if (w[j][k] <= i)//背包当前w容量要>=当前物品的w
                {
                    dp[i] = max(dp[i], dp[i - w[j][k]] + v[j][k]);
                }
             }
        }
```

```
    }
    cout << dp[m]<<'\n';



}
signed main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);
    int t=1;
    while(t--)
        solve();
    return 0;
}
```

B River Crossing S 洛谷 P2904

思路

一共有n头一共有n头牛需要过河，人过河需要M时间，加上i头牛需要加上M_1 + M_2 + ... + M_i时间（1 <= n <= 2500, 1 <= m <= 1000），要求求过河的最短时间。 设计dp[i]为运送完i头牛所需要的最短时间，初始化dp[i] = M + M_1 + ... + M_i。对于一头牛的运送，dp[1]肯定为M + M_1；对于两头牛，我可以选择一口气全部晕过去，也可以选择先采用运一头的最短时间，然后再回来再运下一头；同样对于三头牛，我可以一口气全部运，也可以先一后二，也可以先二后一，以此类推。 所以dp[i]的转移式子为dp[i] = max(dp[i], dp[j] + M + M + M_1 + ... + M_(i-j))，因为要回来所以有两个M，j为从1到i-1的遍历。 最坏情况下计算dp[i]从1到n需要遍历一遍，其中j从1到i-1还需要遍历一遍，复杂度为n方最坏情况下计算dp[i]从1到n需要遍历一遍，其中j从1到i-1还需要遍历一遍，复杂度为O（n^2）。

代码

```
    #include <bits/stdc++.h>
    using namespace std;

    int main()
    {
        ios::sync_with_stdio(0);
        cin.tie(0), cout.tie(0);

        int n, M;
        cin >> n >> M;

        vector<int> arr(n + 1);
        for (int i = 1; i <= n; i++)
        {
            cin >> arr[i];
        }

        vector<int> M_i(n + 1, 0); // 前缀和求M_i
```

```
        for (int i = 1; i <= n; i++)
        {
            M_i[i] = M_i[i - 1] + arr[i];
        }

        vector<int> dp(n + 1, 0); // 初始化dp, dp[i]为i只牛运过河的最短时间
        for (int i = 1; i <= n; i++)
        {
            dp[i] = M + M_i[i];
        }

        for (int i = 2; i <= n; i++) // 顺序
        {
            for (int j = 1; j < i; j++)
            {
                dp[i] = min(dp[i], dp[j] + M_i[i - j] + 2 * M); // 从头遍历
dp[i] 为 dp[j] + M_i[i-j](还需要云i-j头) + 2*M（人需要回去再渡河）
            }
        }

        cout << dp[n] << "\n";

        return 0;
    }
```

C Writing Code CodeForces 543A

思路

题目中选m个程序员且bug少于b个，相当于是二维背包问题。如果将程序员作为物体，则问题可以描述为：有n种物品，每种物品体积为1,重量为a~1~。要求选则体积恰好为m且重量小于b，求方案数。 所以开三维数组dp~i,j,k~，表示前i种物品，体积为j，重量为k的方案数。状态转移方程与完全背包相似， 即dp~i,j,k~ += dp~i,j-1,k-a[i]~。 但是三维数组会MLE所以优化掉第一维。

代码

```
#include <bits/stdc++.h>
using namespace std;

int main()
{
    const int maxn = 500;

    int n, m, b, mod;
    cin >> n >> m >> b >> mod;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; ++i)
        cin >> a[i];

    static int dp[maxn + 1][maxn + 1];
```

```
    dp[0][0] = 1;

    for (int i = 1; i <= n; ++i)
        for (int j = 1; j <= m; ++j)
            for (int k = 0; k <= b; ++k)
                if (k >= a[i])
                {
                    dp[j][k] += dp[j - 1][k - a[i]];
                    dp[j][k] %= mod;
                }
    int res = 0;

    for (int i = 0; i <= b; ++i)
    {
        res += dp[m][i];
        res %= mod;
    }

    cout << res;
    return 0;
}
```

D Boredom CodeForces 455A

思路

这种规定元素小于等于1e5 1e6的都可以考虑从值域出发 dp[i]代表考虑到值小于等于i时 能产生的最大分数 这题状态转移方程很明显 取了i位置的元素就不能取i+1和i-1位置的元素了 故dp[i] 要么等于取完所有等于i的+dp[i-2]的最优贡献值(因为取了i就不能取i-1) 要么直接等于dp[i-1] 即不取i位置

代码

```
#include<bits/stdc++.h>
#define int long long
using namespace std;
typedef long long ll;
typedef long double ld;
const int N=100010;//题目规定最大元素小于等于1e5

void solve()
{
    int n;
    cin>>n;
    vector<int> num(N);
    for(int i=1;i<=n;i++)
    {
        int t;
        cin>>t;
        num[t]++;
    }
```

```
    vector<int> dp(N);
    dp[1]=num[1];//初始化 代表考虑元素小于等于1的最大分数
    for(int i=2;i<=100000;i++)
    {
        dp[i]=max(dp[i-1],dp[i-2]+num[i]*i);//取一个元素一定取完最优
    }
    cout<<dp[100000]<<endl;



}
signed main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);
    int t=1;
    while(t--)
        solve();
    return 0;
}
```

E 石子合并 洛谷 P1880

思路

环形摆放的n堆石子，每次选相邻的两堆合并成新的一堆，得分为合并后的石子个数(0 <= ai <= 100)，求全部合并成一堆的得分的最大值和最小值（1 <= n <= 100）。 环形的n堆石子合并成一堆，在解的条件下，我们肯定能找到一个断点，使得圆转换成链，此时不用考虑环形就能得到答案。因此我们重复两遍石子序列来模拟寻找断点。 设计dp[i][j][k]，k的取值为0、1，分别表示将[i, j]石子堆合并成一堆的最小积分和最大积分。当i = j时，此时的积分为0，初始化为0。 始终坚持两个一堆合并成一个大的堆，因此在求dp[i][j][k]的时候我们需要先从j-i值较小的地方开始合并。定义一个mid，遍历i到j，选择合并后积分最小/大的两堆进行合并记录到dp[i][j][0/1]上。因此转移方程为，dp[i][j][0/1] = min(dp[i][j][0/1], dp[i][mid][0/1] + dp[mid+1][j][0/1] + sum)。这个sum是i到j所有元素的和，是此次合并所得的积分，因为是连续的一串数的和，所以可以用前缀和来维护。 最后因为我们求的是1到2*n长度的值，我们需要的是1到n的最大值和最小值，所以遍历dp[i][i+n-1][0/1]取最大值和最小值。一共有3层循环结构，复杂度为O（n^3）。

代码

```
#include <bits/stdc++.h>
using namespace std;

int main()
{
    ios::sync_with_stdio(0);
    cin.tie(0), cout.tie(0);

    int n;
    cin >> n;
```

```
    vector<int> arr(2 * n + 1); // 因为是环形，将数据重复两边模拟环
    for (int i = 1; i <= n; i++)
    {
        cin >> arr[i];
        arr[i + n] = arr[i];
    }

    vector<int> pre_arr(2 * n + 1, 0); // 维护前缀和, 减少复杂度
    for (int i = 1; i <= 2 * n; i++)
    {
        pre_arr[i] = pre_arr[i - 1] + arr[i];
    }

    int dp[2 * n + 1][2 * n + 1][2]; // 设计dp， dp[i][j][x]表示i到j合并成一堆，dp[i][j][0]记录最小值, dp[i][j][1]记录最大值
    for (int i = 1; i <= 2 * n; i++)
    {
        dp[i][i][0] = 0; // 只有一堆就是0
        dp[i][i][1] = 0;
    }

    for (int k = 1; k <= 2 * n; k++)
    {
        for (int i = 1; i + k <= 2 * n; i++)
        {
            dp[i][i + k][0] = 1e9; // 初始化
            dp[i][i + k][1] = 0;

            int sum = pre_arr[i + k] - pre_arr[i - 1]; // 通过前缀和计算此次合并需要增加的积分

            for (int j = i; j < i + k; j++) // 遍历从i到i+k，意味着将（i-j）和（j+1, i+k）合并成一堆
            {
                dp[i][i + k][0] = min(dp[i][i + k][0], dp[i][j][0] + dp[j + 1][i + k][0] + sum);
                dp[i][i + k][1] = max(dp[i][i + k][1], dp[i][j][1] + dp[j + 1][i + k][1] + sum);
            }
        }
    }

    int my_min = 1e9, my_max = 0;
    for (int i = 1; i + n - 1 <= 2 * n; i++) // 处理环形结构，取dp长度为n的值取最小、大值
    {
        my_min = min(my_min, dp[i][i + n - 1][0]);
        my_max = max(my_max, dp[i][i + n - 1][1]);
    }

    cout << my_min << "\n"
         << my_max << "\n";
```

```
    return 0;
}
```

F 吃奶酪 洛谷 P1433

思路

N=15这个数据量 可以考虑搜索剪枝 状压dp等方法 这里采用状压dp 定义dp[i][j]为老鼠现在在i奶酪处 吃完的奶酪集合表示为j的最小距离 下面讲解一下状压dp的集合表示 其实就是用二进制的第i位来代表第i个奶酪有没有吃 比如1011 就是吃了第1 2 4个 1010就是吃了第2 4个奶酪 核心就是把一个较为复杂 不好表示的状态用一个简单的数字表示 下面是状态转移 一般地 考虑如何转移得到在i个奶酪状态集合为k的最短距离 假设我们要求状态集合为k时 终点为i 即dp[i][k] 如何得到？ 显然目前终点为i i这一位在k里一定为1 而且是最后一个被访问 设j在k集合里是访问过的 则dp[j][k^(1<<i)]是终点为j的 没有访问过i的最小距离 这个状态的老鼠如果从j走到i 就是dp[i][k]的状态 这样遍历每一个j 取min就能求出dp[i][k]可能的最短距离啦

代码

```
#include<bits/stdc++.h>
#define int long long
using namespace std;
typedef long long ll;
typedef long double ld;
const int maxn=(1<<16);

int n;
struct node
{
    ld x;
    ld y;
};
ld dis(node x,node y)//两点距离
{
    return sqrt((x.x-y.x)*(x.x-y.x)+(x.y-y.y)*(x.y-y.y));
}
void solve()
{
    cin>>n;
    vector<node> nodes(n);
    for(int i=0;i<n;i++)
        cin>>nodes[i].x>>nodes[i].y;
    int N=(1<<n);
    vector<vector<ld>> dp(n,vector<ld>(N,0x3f3f3f3f));//也可以采用
memset(dp,127,sizeof(dp));为浮点数赋极大值
    for(int i=0;i<n;i++)
    {
        node tmp={0,0};
        dp[i][1<<i]=dis(tmp,nodes[i]);
    }
    for(int k=0;k<N;k++)
    {
```

```
        for(int i=0;i<n;i++)
        {
            if(k&(1<<i))//确保第i位是1
            {
                for(int j=0;j<n;j++)
                {
                    if(k&(1<<j))
                        dp[i][k]=min(dp[i][k],dp[j]
[k^(1<<i)]+dis(nodes[i],nodes[j]));//状态转移
                }
            }
        }
    }
    ld ans=0x3f3f3f3f;
    for(int i=0;i<n;i++)
        ans=min(ans,dp[i][N-1]);
    cout<<fixed<<setprecision(2)<<ans<<'\n';//设置为2位小数




}
signed main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);
    int t=1;
    //cin>>t;
    while(t--)
        solve();
    return 0;
}
```

G Cows in a Skyscraper G 洛谷 P3052

思路

首先看到N小于等于18,可以考虑状压dp。 数组dp~i,j~表示使用i辆电梯装j情况奶牛时第i辆电梯已有重量，使用j的二进制形式表示奶牛的装载情况，j的第k位为1则说明第k个奶牛已经装上了。 状态转移时，首先遍历i，从少到多遍历电梯，然后遍历j，表示所有的装载情况。如果dp~i，j~不是无效值则考虑增加更多的奶牛。 增加奶牛即进行状态转移，按照能否在第i个电梯装下奶牛分类。可以的话就更新dp~i,j~的值，否则更新dp~i+1,j~。 遍历结束后找到j全为1且dp~i,j~不为无效值的最小i即为答案。

代码

```
#include <bits/stdc++.h>
using namespace std;

int main()
```

```
{
    int n, w;
    cin >> n >> w;

    vector<int> c(n);
    for (int &x : c)
        cin >> x;

    int maxn = 1 << n;

    vector<vector<int>> dp(n + 1, vector<int>(maxn, INT_MAX));

    for (int i = 0; i < n; i++)
        dp[1][1 << i] = c[i];

    for (int i = 1; i <= n; ++i)
        for (int j = 0; j < maxn; ++j)
            if (dp[i][j] != INT_MAX)
                for (int k = 0; k < n; ++k)
                {
                    if ((j & (1 << k)) != 0)
                        continue;

                    if (dp[i][j] + c[k] <= w)
                        dp[i][j | (1 << k)] = min(dp[i][j | (1 << k)],
dp[i][j] + c[k]);
                    else
                        dp[i + 1][j | (1 << k)] = min(dp[i][j | (1 << k)],
c[k]);
                }

    for (int i = 1; i <= n; i++)
    {
        if (dp[i][maxn - 1] != INT_MAX)
        {
            cout << i << endl;
            break;
        }
    }

    return 0;
}
```

另解

这题用迭代加深dfs+剪枝也能跑过，理论复杂度是O(n*n!)，但是实际上跑的飞快。 对奶牛重量倒序排序，并在dfs时剪掉奶牛位数大于车厢位数的部分。

代码

```cpp
#include <bits/stdc++.h>
using namespace std;

bool dfs(int x, int n, vector<int> &c, vector<int> &cnt, int w)
{
    if (x == n + 1)
        return true;

    for (int i = 1; i < cnt.size() && i <= x; ++i)
        if (c[x] + cnt[i] <= w)
        {
            cnt[i] += c[x];
            if (dfs(x + 1, n, c, cnt, w))
                return true;
            cnt[i] -= c[x];
        }

    return false;
}

int main()
{
    int n, w;
    cin >> n >> w;

    vector<int> c(n + 1);
    for (int i = 1; i <= n; ++i)
        cin >> c[i];

    sort(c.begin() + 1, c.end(), greater<int>());

    for (int i = 1; i <= n; ++i)
    {
        vector<int> cnt(i + 1);
        if (dfs(1, n, c, cnt, w))
        {
            cout << i << endl;
            break;
        }
    }

    return 0;
}
```

H FT Robot AtCoder arc087_b

思路

机器人初始在（0， 0），朝x的正方向，有一个由F、T组成的字符串指令（F为向前走一格，T为顺/逆时针旋转90度），问最终是否可能到达给定点（x， y）。(|s| <= 8000) 如果直接dfs，那么每次遇到T都需要分顺时针和逆时针两种情况，最坏为2^8000，显然是不可能的。我们可以采用dp，以T分割每一轮，通过dp来避免多次遍

历同一轮相同点。但是如果直接记录每一轮的坐标，需要空间（28000）（28000）n(轮数)，在最坏情况下肯定会MLE。所以我们考虑把二维转成一维，分别考虑cur_x, cur_y能否到达最终的x，y，奇数轮是x轴上的运动，偶数论是y轴上的运动，方向除了x的第一轮之外，可以选择正或者负。 定义dp_x[i][j]，i表示第i论，j表示坐标，表示完成第i轮后，机器人可能到达的点（值为1说明可以到达，值为0说明不能到达）。转移方程为dp[i][j] ? (dp[i+1][j - arr_x[i+1]] = 1, dp[i+1][j + arr_x[i+1]] = 1 :)。y轴上的运动也是同理，需要注意的是x的第一轮方向固定，要特殊判断一下。 特别提示一下需要空间最大可能为4000（x或y最大可能的轮数） * （8000 * 2）（需要的坐标轴空间）* 2（x，y两个）在MLE边缘，在申请坐标轴空间的时候不要有太多用不上的空间。复杂度为$O(n^2)$。

代码

```
#include <bits/stdc++.h>
using namespace std;

int main()
{
    ios::sync_with_stdio(0);
    cin.tie(0), cout.tie(0);

    string str;
    cin >> str;

    int x, y;
    cin >> x >> y;

    vector<int> arr_x, arr_y;

    int flag = 0;

    int cnt = 0;
    for (int i = 0; i < str.size(); i++) // 将字符分为上下走和左右走
    {
        if (str[i] == 'F')
        {
            cnt++;
        }
        else
        {
            if (flag == 0)
            {
                arr_x.push_back(cnt);
            }
            else
            {
                arr_y.push_back(cnt);
            }

            cnt = 0;
            flag ^= 1;
        }
    }
```

```
    if (flag == 0) // 最后剩下的一部分
    {
        arr_x.push_back(cnt);
    }
    else
    {
        arr_y.push_back(cnt);
    }

    int len_x = arr_x.size(); // 对左右进行dp

    int dp_x[len_x + 1][16001];
    dp_x[0][8000] = 1;
    for (int i = 0; i < arr_x.size(); i++)
    {
        if (i == 0) // 第一步必须向x轴正方向走
        {
            dp_x[i + 1][8000 + arr_x[i]] = 1;
            continue;
        }

        for (int j = 0; j < 16000; j++)
        {
            if (dp_x[i][j] == 1)
            {
                dp_x[i + 1][j + arr_x[i]] = 1;
                dp_x[i + 1][j - arr_x[i]] = 1;
            }
        }
    }

    if (!dp_x[len_x][x + 8000])
    {
        cout << "No" << "\n";
        return 0;
    }

    int len_y = arr_y.size();

    int dp_y[len_y + 1][16001];
    dp_y[0][8000] = 1;
    for (int i = 0; i < arr_y.size(); i++)
    {
        for (int j = 0; j < 16000; j++)
        {
            if (dp_y[i][j] == 1)
            {
                dp_y[i + 1][j + arr_y[i]] = 1;
                dp_y[i + 1][j - arr_y[i]] = 1;
            }
        }
    }
```

```
    if (!dp_y[len_y][y + 8000])
    {
        cout << "No" << "\n";
        return 0;
    }

    cout << "Yes" << "\n";

    return 0;
}
```

I 货币系统 洛谷 P5020

思路

题目最重要的是想到B是A的子集，可以使用反证法证明： 假设存在 $ x \subseteq B $， 且 $ x \nsubseteq A $ ，则x一定可以被A中的某几个元素表示。这些元素一定在B中否则B无法表示A可以表示的数值，继而x就是多余元素可以删去。 已知B是A的子集后要求最小化B，就是要剔除A中多余元素。多余元素就是可以被其他元素表示的元素。 使用f~i~表示值为i的数能否被A中的元素表示。 从小到大遍历A，如果首次遍历A~i~时f~a[i]~==1，则说明A~i~可以被比他小的数表示，所以剔除A~i~。否则标记更大的数，此处的思想类似于筛法.

代码

```
#include <bits/stdc++.h>
using namespace std;

int main()
{
    int t;
    cin >> t;
    while (t--)
    {
        int n;
        cin >> n;
        vector<int> a(n);

        for (int &x : a)
            cin >> x;

        sort(a.begin(), a.end());
        vector<int> f(a[n - 1] + 1);

        f[0] = 1;
        int res = n;

        for (int i = 0; i < n; ++i)
        {
            if (f[a[i]])
            {
                --res;
```

```
                continue;
            }

            for (int j = a[i]; j < f.size(); ++j)
                f[j] = f[j] | f[j - a[i]];
        }

        cout << res << endl;
    }

    return 0;
}
```

J problem a 洛谷 P2519

思路

先转化题意 有ai个人成绩更高 bi个人成绩更低 其实就相当于i的排名属于[ai+1,bi-1]这个区间 最少有几个人说假话不好求 转化为求最多说真话的 分类讨论 显然a+b>=n-1肯定是假话 两个人的排名区间不完全重合肯定是假话 比如[a,b] [b,c] 这样的话a b c一定同分 则[a,b]区间应该是[a,c]才对 然后进一步将问题转化成区间问题 将每个区间赋一个权值w[i]=min(相同区间数,区间长度) 这个w代表取这个区间能得到的说真话的人数 且相同人数不能大于长度了 问题就转化为 有若干个区间，每个区间都有一个权值，要求选出若干不相交区间使得权值和最大。可通过 F[i]=max(F[i],F[left]+value)，left 是这个区间的左端点，value 是这个区间的权值 转移解决

代码

```
#include<bits/stdc++.h>
#define int long long
using namespace std;
typedef long long ll;
typedef long double ld;
const int maxn=2e5;

int tot=0;
int n;
int w[maxn];
int L[maxn];
int R[maxn];
int cnt;
struct line
{
    int l;
    int r;
}lines[maxn];
bool cmp(line x,line y)
{
    if(x.r!=y.r)
        return x.r<y.r;//关键字：右端点
    return x.l<y.l;//次关键字：左端点
}
```

```
void solve()
{
    int n;
    cin>>n;
    for(int i=1;i<=n;i++)
    {
        int a,b;
        cin>>a>>b;
        if(a+b>=n)
            continue;
        lines[++tot].l=a+1;
        lines[tot].r=n-b;
    }
    sort(lines+1,lines+tot+1,cmp);
    int num=0;
    for(int i=1;i<=tot;i++)//计算区间权值
    {
        if(lines[i].l!=lines[i-1].l||lines[i].r!=lines[i-1].r) num++;
        //如果线段完全重合则num值不发生变化，权值增加
        w[num]=min(w[num]+1,lines[i].r-lines[i].l+1);//区间权值
        L[num]=lines[i].l,R[num]=lines[i].r;//记录重新选出来的线段
    }
    vector<int> f(n+1);//f[i]代表仅考虑到i的最大权值
    int j=1;
    for(int i=1;i<=n;i++)
    {
        f[i]=f[i-1];
        while(j<=num&&R[j]==i)
        {
            f[i]=max(f[i],f[L[j]-1]+w[j]);
            j++;
        }
    }
    cout<<(n-f[n])<<'\n';//转化会最少说假话的人
}
signed main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);
    int t=1;
    //cin>>t;
    while(t--)
        solve();
    return 0;
}
```

K 同类分布 洛谷 P4127

思路

求出[a, b]中（原数%各位数之和=0）的数的个数（1 <= a <= b <= 1e18）；计算[a, b]，我们可以拆解成[0, b] - [0, a-1]，这样可以避免很多特殊判断。由于a，b的取值都非常大，直接遍历时不可能的，我们就需要去考虑那些地方是类似可以不用重复计算的。可以发现类似于7000，8000.9000，当它对后面每一位的取值都没有限制的时候，它后面的遍历都是1到999。因此我们可以设计dp[i][j][k]，i表示目前已经完成了第i位，j表示当前前i位数的和，k表示前i位数所构成的原数。此外，我们还需要定义一个limit，用来标志当前这一位的取值有无限制（如果前面的每一位都和a/b一致那就是被限制了，这一位最大只能取a/b这一位上的值；如果没有被限制，那么该位可以取0到9）。 但是dp[i][j][k]中k的取值与a，b的取值相关，a，b的取值非常大的时候会爆空间。这时候我们考虑能不能引进一个mod，来约束k的大小，但是不影响条件判断。当mod为一个定值的时候，我们最后求出的原数是很难判断是否能整除各位数和的，所以mod是一个变的值，最好和各位数和相关。可以发现，a/b的取值最大有1e18，最多有18位，那么数位和最多就是0到160左右。共1e18的数可以分到这一百多种情况下，将这些数位和作为mod，我们就不需要担心k太大以及条件不好判断的问题，最后直接判断sum是否为mod且原数%mod是否为0。分别考虑不同mod时的计数，最后相加。 整个的复杂度是O（len^4）。len是数字的长度，这个在len<=19的情况下是可以跑的。

代码

```
#include <bits/stdc++.h>
using namespace std;

#define int long long

int len;
int arr[20];

int mod;
int dp[20][200][200]; // 标准的数位dp，设计dp[i][j][k], i表示当前为第i位, j表示当前数位和为j, k表示当前原来的数字取模mod后的结果（原数字太大）

int dfs(int pos, int sum, int x, int limit) // pos表示当前位，sum表示当前数位和，x表示当前原数取模，limit表示对当前位的取值是否有限制（0，1）
{
    if (pos > len) // 边界判断
    {
        if (sum == 0) // 全为0直接返回0
        {
            return 0;
        }

        if (sum == mod && x % mod == 0)
        {
            return 1;
        }

        return 0;
    }

    if (limit == 0 && dp[pos][sum][x] != -1) // 记忆化搜索，只有对当前及以后位置无限制的时候才可以返回已经记录的值
    {
        return dp[pos][sum][x];
```

```
    }

    int up = arr[len - pos + 1]; // 从最高位开始，因此为len-pos+1
    if (limit == 0)
    {
        up = 9; // 无限制当前位最高可以取9
    }

    int res = 0;

    for (int i = 0; i <= up; i++)
    {
        int next_sum = sum + i;
        int next_x = (x * 10 + i) % mod;
        int next_limit = 0;

        if (limit && i == up) // 只有当limit有限制，并且当前也取到限制的时候限制才
继续
        {
            next_limit = 1;
        }

        res += dfs(pos + 1, next_sum, next_x, next_limit);
    }

    if (limit == 0)
    {
        dp[pos][sum][x] = res;
    }

    return res;
}

int count(int x) // 用来计算0-x中满足条件的个数
{
    len = 0;
    while (x != 0) // 将x存入数组arr中，arr[1]为个位数
    {
        len++;
        arr[len] = x % 10;
        x /= 10;
    }

    int res = 0;                          // mod值一定要变化，不然计数的条件不好
判断
    for (mod = 1; mod <= 9 * len; mod++) // 遍历所有可能的mod
    {
        memset(dp, -1, sizeof(dp)); // 重置dp状态

        res += dfs(1, 0, 0, 1);
    }

    return res;
}
```

```
signed main()
{
    ios::sync_with_stdio(0);
    cin.tie(0), cout.tie(0);

    int l, r;
    cin >> l >> r;

    cout << (long long)(count(r) - count(l - 1)) << "\n";

    return 0;
}
```

L A Twisty Movement CodeForces 933A

思路

a~i~仅能为1或2,那么不下降子序列只能是 [x*1][y*2] 为了得到上面的目标序列那么反转之前的原始序列应该是 [a*1][b*2][c*1][d*2]，反转中间两部分得到目标序列。所以只要求这个序列的最大长度。 所以就可以用一个 dp，其中dp~i,1~表示[a*1]的最大长度，dp~i,2~表示[a*1][b*2]的最大长度，以此类推。 状态转移：每种序列的最大长度可以继承前一种序列的长度或者延长当前序列的最大长度。

代码

```
#include <bits/stdc++.h>
using namespace std;

int main()
{
    int n;
    cin >> n;
    vector<int> a(n + 1);

    for (int i = 1; i <= n; ++i)
        cin >> a[i];

    vector<vector<int>> dp(n + 1, vector<int>(5));

    for (int i = 1; i <= n; ++i)
    {
        dp[i][1] = dp[i - 1][1] + (a[i] == 1);
        dp[i][2] = max(dp[i][1], dp[i - 1][2] + (a[i] == 2));
        dp[i][3] = max(dp[i][2], dp[i - 1][3] + (a[i] == 1));
        dp[i][4] = max(dp[i][3], dp[i - 1][4] + (a[i] == 2));
    }

    cout << dp[n][4];
```

```
    return 0;
}
```

M Kaavi and Magic Spell CodeForces 1336C

思路

这道题容易想到区间dp 因为既可以从后面加 也可以从前面加 可以看作区间的扩大 比较难想的点在于状态定义 令dp[i][j]为有多少种方案能使得S1-i通过插入使得前min(tlen,j-i+1)个字符构成Ti到Tmin(tlen,j-i+1) 首先是初始化显然 当i<=lent且s[1]==t[i]时 dp[i][i]=2(头插 尾插) 当i>lent时 dp[i][i]=2; 因为此时没有要求 故怎么插入都可以 然后是状态转移 转移过程中 设i为当前A串长度 j为左端点 显然有两种转移方式 先考虑头插 当j>tlen时 此时没有约束 直接插入就好 当j<=tlen时 此时该状态能从dp[j+1][j+i-1]转移当且仅当s[i]=t[j] 再是尾插 当j+i-1>tlen时 显然可直接插入 当j+i-1<=tlen时 此时状态能从dp[j][j+i-2]转移当且仅当t[j+i-1]==s[i]; 形式化的 f[j][i + j - 1] += f[j + 1][i + j - 1] 当 (t[j] == s[i] 或 j > tlen)

f[j][i + j - 1] += f[j][j + i - 2] 当 (t[i + j - 1] == s[i] 或 i + j - 1 > tlen)

如果不能理解 可以把A想象成一个双端队列 不断往里面加元素的过程

代码

```
#include<bits/stdc++.h>
#define int long long
using namespace std;
typedef long long ll;

const int maxn=2e5;
const int mod=998244353;
void solve()
{
    string s,t;
    cin>>s>>t;
    int slen=s.size();
    int tlen=t.size();
    s=" "+s;
    t=" "+t;//从0_based转化成1_based 便于区间dp
    vector<vector<int>> dp(slen+1,vector<int>(slen+1));
    //dp数组初始化
    for(int i=1;i<=slen;i++)
    {
        if(i>tlen)
            dp[i][i]=2;
        else if(s[1]==t[i])
            dp[i][i]=2;
    }
    //状态转移
    //填表法 枚举未知 再用已知信息推未知
    for(int i=2;i<=slen;i++)
    {
        for(int j=1;j+i-1<=slen;j++)
```

```
            {
                if(j>tlen||s[i]==t[j])
                    dp[j][j+i-1]=(dp[j][j+i-1]+dp[j+1][j+i-1])%mod;
                if(j+i-1>tlen||t[j+i-1]==s[i])
                    dp[j][j+i-1]=(dp[j][j+i-1]+dp[j][j+i-2])%mod;
            }
        }
        int ans=0;//答案是A长度为tlen到slen的和
        for(int i=tlen;i<=slen;i++)
        {
            ans=(ans+dp[1][i])%mod;
        }
        cout<<ans<<'\n';


    }
    signed main()
    {
        ios::sync_with_stdio(false);
        cin.tie(nullptr);
        cout.tie(nullptr);
        int t=1;
        //cin>>t;
        while(t--)
            solve();
        return 0;
    }
```

N Coloring Brackets CodeForces 149D

思路

给括号上色，相匹配的括号，一个上红/蓝，另一个上黑色，相邻的括号不能是相同的彩色，求方案数。 我们不妨令0为黑色，1为蓝色，2为红色，那么一对括号的颜色组合为（1，0）（2，0）（0，1）（0，2）。一对括号的颜色是受它的两边的括号和被它包含的括号还有包含它的括号所影响的，所以我们不妨设计dp[l][cl][cr]，l表示左括号编号为l的一组括号所包括的区域，cl为该左括号的颜色，cr为该右括号的颜色。转移公式往括号之间的颜色冲突去想。 观察括号字符串，不难得到一个森林结构。与它想连的就是它的兄弟节点，被它包括的就是它的儿子节点，包含它的就是它的父亲节点。根据这个建树，最后没有父亲节点的我们统一连接到0节点。 接下来我们考虑怎么把儿子节点的dp[son][cl][cr]转换成父亲节点的dp[father][cl][cr]，如果一个节点没有子节点（叶子节点），那么四种涂色方案的都为1。对于(()()……())这种结构，对于第一个儿子，我们要考虑左边框的颜色进行上色，对于其他儿子，我们要考虑前一个儿子的右边框进行上色。因此我们定义一个新的状态转移G[i][c_last]，i表示完成了第i个儿子的上色任务，c_last表示当前最后一个括号的上色情况。假设上一个儿子上色完成后的其中一种情况是颜色为黑色，那么当前儿子支持（0，1）（1，0）（2，0）（0，2），这些上色选择又分别导入不同的G[i+1][0/1/2]。最后完成全部儿子的上色G[len][0/1/2]后还需要考虑父亲右边框是否会冲突。具体的转移方程我展开写到代码里。 就这样层层向上转移，需要注意的是，我们还定义了一个0节点作为所有节点的根节点。把所有树指向0，相当于在整个括号序列里左右两边各自加上一对颜色为黑色的括号。我在代码里做了特判。最后返回的dp[0][0][0]就是答案，记得取模。 复杂度为O（n）。

代码

```cpp
#include <bits/stdc++.h>
using namespace std;

#define int long long

const int mod = 1e9 + 7; // 取模
const int MAXN = 705;    // 字符串的最大长度

int dp[MAXN][3][3];                                  // dp[i][j][k]，i表示左边框编号为i的配对，j表示左边框的颜色，k表示右边框的颜色（0为黑色，1为蓝色，2为红色）
vector<int> comp(MAXN, -1);                          // comp[i]表示与i配对的另一字符编号
vector<vector<int>> tree(MAXN, vector<int>()); // 建树，tree[i]中的内容为i的子节点

int choice[4][2] = {{0, 1}, {1, 0}, {2, 0}, {0, 2}}; // 四种配对括号的颜色填充

void dfs(int l)
{
    int len = tree[l].size();

    if (tree[l].size() == 0) // 说明当前节点为叶子节点，直接返回1
    {
        dp[l][1][0] = 1;
        dp[l][0][1] = 1;
        dp[l][2][0] = 1;
        dp[l][0][2] = 1;

        return;
    }

    for (int i = 0; i < tree[l].size(); i++) // 后序遍历，先遍历子节点，得到子节点的dp[i][j][k]
    {
        dfs(tree[l][i]);
    }

    int G[len + 1][3];        // 兄弟节点之间的涂色也用dp解决，G[i][j]，i表示已经完成了前i个任务，j表示第i个任务最后一个括号的颜色
    memset(G, 0, sizeof(G)); // 需要维护最后一个括号的颜色以便进行下个任务的完成，或者判断是否能接到父亲节点的右括号上

    if (l == 0) // 当l为0时，特判一下，因为实际上没有0这个点，所以假设整个字符串被一组黑框包括，因此答案为dp[0][0][0]
    {
        for (int i = 0; i < tree[l].size(); i++)
        {
            int now = tree[l][i];

            if (i == 0) // 第一个任务需要特判
            {
                G[i + 1][0] = dp[now][1][0] + dp[now][2][0];
                G[i + 1][1] = dp[now][0][1];
```

```
                G[i + 1][2] = dp[now][0][2];

                continue;
            }

            G[i + 1][0] = (G[i][0] * (dp[now][1][0] + dp[now][2][0])) % mod
+ // G[i+1][0]意味着当前任务完成后末尾颜色为黑色
                          (G[i][1] * dp[now][2][0]) % mod +
// 由G[i][0]连接上(1, 0),(2, 0)，G[i][1]
                          (G[i][2] * dp[now][1][0]) % mod;
// 连接上(2, 0)，G[i][2]连接上(1, 0)
            G[i + 1][0] %= mod;
// 剩下两种情况以此类推

            G[i + 1][1] = (G[i][0] * dp[now][0][1]) % mod +
                          (G[i][1] * dp[now][0][1]) % mod +
                          (G[i][2] * dp[now][0][1]) % mod;
            G[i + 1][1] %= mod;

            G[i + 1][2] = (G[i][0] * dp[now][0][2]) % mod +
                          (G[i][1] * dp[now][0][2]) % mod +
                          (G[i][2] * dp[now][0][2]) % mod;
            G[i + 1][2] %= mod;
        }

        dp[l][0][0] = G[len][0] + G[len][1] + G[len][2]; // 最后完成了len个任
务后的G[len][j]还需要考虑父节点右边框的颜色，不能冲突
        dp[l][0][0] %= mod;

        return;
    }

    for (int k = 0; k < 4; k++)
    {
        int left_c = choice[k][0];
        int right_c = choice[k][1];

        for (int i = 0; i < tree[l].size(); i++)
        {
            int now = tree[l][i];

            if (i == 0)
            {
                if (left_c == 0)
                {
                    G[i + 1][0] = dp[now][1][0] + dp[now][2][0];
                    G[i + 1][1] = dp[now][0][1];
                    G[i + 1][2] = dp[now][0][2];
                }
                else if (left_c == 1)
                {
                    G[i + 1][0] = dp[now][2][0];
                    G[i + 1][1] = dp[now][0][1];
                    G[i + 1][2] = dp[now][0][2];
```

```
                }
                else if (left_c == 2)
                {
                    G[i + 1][0] = dp[now][1][0];
                    G[i + 1][1] = dp[now][0][1];
                    G[i + 1][2] = dp[now][0][2];
                }

                G[i + 1][0] %= mod;
                G[i + 1][1] %= mod;
                G[i + 1][2] %= mod;

                continue;
            }

            G[i + 1][0] = (G[i][0] * (dp[now][1][0] + dp[now][2][0])) % mod
+
                          (G[i][1] * dp[now][2][0]) % mod +
                          (G[i][2] * dp[now][1][0]) % mod;
            G[i + 1][0] %= mod;

            G[i + 1][1] = (G[i][0] * dp[now][0][1]) % mod +
                          (G[i][1] * dp[now][0][1]) % mod +
                          (G[i][2] * dp[now][0][1]) % mod;
            G[i + 1][1] %= mod;

            G[i + 1][2] = (G[i][0] * dp[now][0][2]) % mod +
                          (G[i][1] * dp[now][0][2]) % mod +
                          (G[i][2] * dp[now][0][2]) % mod;
            G[i + 1][2] %= mod;
        }

        if (right_c == 0) // 根据右边框颜色，选择不会冲突的颜色相加
        {
            dp[l][left_c][right_c] = G[len][0] + G[len][1] + G[len][2];
        }
        else if (right_c == 1)
        {
            dp[l][left_c][right_c] = G[len][0] + G[len][2];
        }
        else if (right_c == 2)
        {
            dp[l][left_c][right_c] = G[len][0] + G[len][1];
        }

        dp[l][left_c][right_c] %= mod;
    }
}

signed main()
{
    ios::sync_with_stdio(0);
    cin.tie(0), cout.tie(0);
```

```
    string str;
    cin >> str;

    stack<int> stk;
    for (int i = 0; i < str.size(); i++) // 建树，定义一对括号被另一对括号包含，则该括号为另一括号的儿子
    {
        if (str[i] == '(')
        {
            stk.push(i + 1);
        }
        else
        {
            int it = stk.top();
            stk.pop();

            comp[it] = i + 1;
            comp[i + 1] = it;

            if (stk.empty()) // 栈为空说明没有父亲节点，连到零上，作为所有树的根
            {
                tree[0].push_back(it);
            }
            else
            {
                tree[stk.top()].push_back(it);
            }
        }
    }

    memset(dp, 0, sizeof(dp));

    dfs(0); // 从最初的节点开始搜索

    cout << dp[0][0][0] << "\n";

    return 0;
}
```

O LCIS CodeForces 10D

思路

首先设dp数组，其中dp~i,j~表示A的前i个元素和B的前j个元素组成且以B~j~结尾的最长公共递增子序列长度。状态转移时，如果 $ A~i~ \neq B~j~ $，则直接继承i-1的答案。 否则，回溯j前的合法结果并找到最优的继承方法。 为了输出路径，使用二维数组记录每个位置的前驱节点。

代码

```
#include <bits/stdc++.h>
using namespace std;
```

```
int main()
{
    int n, m;
    cin >> n;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    cin >> m;
    vector<int> b(m + 1);
    for (int i = 1; i <= m; i++)
        cin >> b[i];

    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    vector<vector<int>> pre(n + 1, vector<int>(m + 1, 0));
    int res = 0, x = 0, y = 0;

    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= m; j++)
        {
            if (a[i] != b[j])
            {
                dp[i][j] = dp[i - 1][j];
                pre[i][j] = pre[i - 1][j];
            }
            else
            {
                int maxv = 0;
                int pos = 0;

                for (int k = 0; k < j; k++)
                {
                    if (b[k] < a[i] && dp[i - 1][k] > maxv)
                    {
                        maxv = dp[i - 1][k];
                        pos = k;
                    }
                }
                dp[i][j] = maxv + 1;
                pre[i][j] = pos;

                if (dp[i][j] > res)
                {
                    res = dp[i][j];
                    x = i;
                    y = j;
                }
            }
        }
    }

    vector<int> path;
    while (y != 0)
```

```
    {
        path.push_back(b[y]);
        y = pre[x][y];
    }
    reverse(path.begin(), path.end());

    cout << res << endl;
    for (int x : path)
        cout << x << " ";
    cout << endl;

    return 0;
}
```

P 花神的数论题 洛谷 P4317

思路

题目要求求1-N所有数二进制1的个数的乘积 由于n太大 不能直接算 所以采用转换的思想 把n看成一个二进制集合 把这个集合按二进制前缀不断划分成更小的子集合 对于每个后缀集合 求出该集合的sum(i) 合并后返回前缀 时间复杂度约为O(logn平方) 感性的说明一下 我们原来要计算每个数的sum(i)都要递归到最后一位 就形成一个深度为k的二叉搜索树 而记忆化之后 很多二叉分支只要走一次了

代码

```
#include<bits/stdc++.h>
#define int long long
using namespace std;
typedef long long ll;
typedef long double ld;

const int mod=1e7+7;
int len;//代表n的二进制长度
ll n;
int a[55];
ll dp[55][55];

//dp[p][st]表示在已经确定前p-1位二进制、其中恰好有st个1，并且后续位不受上界限制的情况下，所有能构造出的整数x的sum(x)的乘积
//其实这里的dp相当于记忆化搜索 避免重复计算

//dfs表示从高位向低位枚举 枚举到第p位 前面有st个1 此位能不能任取的答案
//比如原数二进制表示是1011 如果第一位取0 后面的就能任取了 因为肯定小于n 取1则反之

ll dfs(int p,int st,int limit)
{
    if(p>len)
        return max(st,1LL);//已经枚举到头了 返回1的个数 要避免1的个数为0的情况
    if(dp[p][st]!=-1&&!limit)
        return dp[p][st];//该状态已经计算过了 直接返回
```

```
    ll ret=1;//代表枚举到第p位 前面有st个1 后面所有sum的乘积
    int res;//res代表该位能取0 还是0 1
    if(limit) res=a[len-p+1];//因为我们存储的时候是从低位到高位的 所以要反一下
    else res=1;
    for(int i=0;i<=res;i++)//对p位赋值
    {
        int tmp;//代表对该位赋值后前缀1的个数
        if(i==1)
            tmp=st+1;
        else
            tmp=st;
        (ret*=dfs(p+1,tmp,limit&&(i==res)))%=mod;//如果本来就limit 该位取值后
还是limit 如果本来！limit 取1才limit
    }
    if(!limit) dp[p][st]=ret;//记忆化搜索
    return ret;
}
void solve()
{
    cin>>n;
    while(n)
    {
        a[++len]=n&1;
        n>>=1;
    }//将n转化成从低位到高位的二进制串
    memset(dp,-1,sizeof dp);//统一赋值为-1 代表未搜索过
    cout<<dfs(1,0,1)<<'\n';//初始状态
}
signed main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);
    int t=1;
    //cin>>t;
    while(t--)
        solve();
    return 0;
}
```

Q Minimal Segment Cover CodeForces 1175E

思路

有n个线段和m个查询（1<= n,m <= 2e5），每次查询问给出的x，y，求最少用几段线段覆盖或根本不可能覆盖。 对于一个x值，我想求从x开始能覆盖到的最远的地方(nxt[x])，那肯定是l<=x的线段中r最大的或可能就是x，贪心地想如果我每次找线段覆盖，都找能覆盖最远的地方的线段，那么此时到达y所需要的线段一定是最少的。如果r的最大值<=x，说明从x开始无法找到更大的覆盖了，就让nxt[x]=x形成一个自环。 ☑ = up[i-1][up[i-1][x]]，意思就是对于x跳2^i步的位置就是x先跳2^(i-1)再跳2^(i-1),具体的实现写在代码上了。从x到y假设需要k段才能到达，根据k的二进制表达，我们可以假设为2^a + 2^b + ... + 2^z。从大到小依次跳步，当up[i][cur_x]

< y时就说明可以跳，更新cur_x = up[i][cur]。需要注意的是，我们需要找到最小的到达y的，那我们倍增就去找差一步到达y的，最后判断能否一步到达y。 复杂度为O（nlogn）

代码

```
#include <bits/stdc++.h>
using namespace std;

const int MAXN = 5e5 + 5; // l与r能覆盖的最大范围

bool cmp(const pair<int, int> &a, const pair<int, int> &b) // 按照l大小进行排序
{
    return a.first < b.first;
}

int main()
{
    ios::sync_with_stdio(0);
    cin.tie(0), cout.tie(0);

    int n, m;
    cin >> n >> m;

    vector<pair<int, int>> arr(n + 1);
    for (int i = 1; i <= n; i++)
    {
        cin >> arr[i].first >> arr[i].second;
    }

    sort(arr.begin() + 1, arr.begin() + 1 + n, cmp);

    vector<int> nxt(MAXN); // nxt[i]表示i能跳到的下一个最远的r
    int flag = -1;
    int top = 1;
    for (int i = 0; i < MAXN; i++)
    {
        while (top <= n && arr[top].first <= i)
        {
            flag = max(flag, arr[top].second);
            top++;
        }

        if (flag <= i)
        {
            nxt[i] = i;
        }
        else
        {
            nxt[i] = flag;
        }
    }
```

```
    vector<vector<int>> up(21, vector<int>(MAXN)); // 利用倍增减少查询的复杂度
    for (int k = 0; k <= 20; k++)
    {
        for (int i = 0; i < MAXN; i++)
        {
            if (k == 0) // 不能跳的时候设置自环
            {
                up[k][i] = nxt[i];
                continue;
            }

            up[k][i] = up[k - 1][up[k - 1][i]];
        }
    }

    while (m--)
    {
        int x, y;
        cin >> x >> y;

        int ans = 0;
        int cur = x;

        for (int k = 20; k >= 0; k--)
        {
            int go = up[k][cur];

            if (go > cur && go < y) // 一定要求go>cur并且go<y, go>cur说明cur处
不是自环
            {                       // go>=y可能是跳入的自环刚好大于等于y，如果直
接计算会多计入自环的圈数
                cur = up[k][cur];
                ans += (1 << k);
            }
        }

        if (up[0][cur] > cur && up[0][cur] >= y) // 最后判断所得cur是否能一次跳
入
        {
            cout << ans + 1 << "\n";
        }
        else
        {
            cout << "-1" << "\n";
        }
    }

    return 0;
}
```

R 解锁屏幕 洛谷 P4460

思路

状压dp 设数组dp~i,j~含义为i情况下以第j个点结尾的方案数 状态转移时，对于情况i，遍历所有未使用的点k，如果该点与结尾点j满足条件4，则将dp[i|k][k]+=dp[i][j]; 统计结果时dp数组，如果情况i中已用的点数大于等于4则累加。

为了快速判断三点之间是否满足条件4，预处理出f数组。 f~i,j~表示i,j两个点的连线跨过的点

代码

```
#include <bits/stdc++.h>
using namespace std;

const int mod = 100000007;

struct point
{
    int x, y;
};

inline bool legal(point p1, point p2, point p3, point p4)
{
    int dx1 = p2.x - p1.x;
    int dy1 = p2.y - p1.y;
    int dx2 = p4.x - p3.x;
    int dy2 = p4.y - p3.y;

    return dy1 * dx2 == dy2 * dx1;
}

int count(int x)
{
    int res = 0;
    while (x)
    {
        res++;
        x &= (x - 1);
    }
    return res;
}
int main()
{
    int n;
    cin >> n;

    vector<point> p(n + 1);
    int lim = 1 << (n + 1);

    vector<vector<int>> dp(lim + 1, vector<int>(n + 1));
    vector<vector<int>> f(n + 1, vector<int>(n + 1));

    int res = 0;
```

```
    for (int i = 1; i <= n; i++)
        cin >> p[i].x >> p[i].y;

    sort(p.begin() + 1, p.end(), [](const point &a, const point &b)
         {if(a.x!=b.x)return a.x<b.x;
            else return a.y<b.y; });

    for (int i = 1; i <= n; i++)
        for (int j = i + 1; j <= n; j++)
            for (int k = i + 1; k <= j - 1; k++)
                if (legal(p[i], p[k], p[k], p[j]))
                {
                    f[i][j] |= (1 << k);
                    f[j][i] |= (1 << k);
                }

    for (int i = 1; i <= n; i++)
        dp[1 << i][i] = 1;

    for (int i = 0; i < (1 << (n + 1)); i++)
        for (int j = 1; j <= n; j++)
            if (i & (1 << j))
                for (int k = 1; k <= n; k++)
                    if ((i & (1 << k)) == 0 && k != j && (i & f[j][k]) ==
f[j][k])
                        dp[i | (1 << k)][k] = (dp[i | (1 << k)][k] + dp[i]
[j]) % mod;

    for (int i = 0; i < (1 << (n + 1)); i++)
        if (count(i) >= 4)
            for (int j = 1; j <= n; j++)
                if (i & (1 << j))
                    res = (res + dp[i][j]) % mod;

    cout << res << endl;
    return 0;
}
```

S Hiking CodeForces 489E

思路

题目要求求总疲劳值除以总舒适度的结果 又是最小化 明显是分数规划 采用二分答案的思想 看最小值mid能不能达到 推出疲劳/舒适的表达式后发现容易用dp求解 设dp[i]为在i过完夜的最小值 递推求出dp[n] 并记录路径即可

代码

```cpp
#include<bits/stdc++.h>
#define int long long
using namespace std;
typedef long long ll;
typedef long double ld;

const int mod=998244353;
const int maxn=1010;
const long double eps=1e-6;
ld x[maxn];
ld b[maxn];
int pre[maxn];
ld dp[maxn];
int cnt;
int n,m;
bool check(ld cur)
{
    for(int i=1;i<=n;i++)
    {
        dp[i]=1e18;
    }
    dp[0]=0.00;

    for(int i=0;i<=n;i++)
    {
        for(int j=i+1;j<=n;j++)
        {
            ld tmp=dp[i]+sqrt(abs(x[j]-x[i]-m))*cur-b[j];
            if(dp[j]>tmp)
            {
                dp[j]=tmp;
                pre[j]=i;
            }
        }
    }
    return dp[n]<0;

}
void solve()
{
    cin>>n>>m;
    for(int i=1;i<=n;i++)
    {
        int t1,t2;
        cin>>t1>>t2;
        x[++cnt]=t1;
        b[cnt]=t2;
    }
    ld l=0;
    ld r=1e6;
    while(abs(r-l)>eps)//浮点数比较有误差 故定义一个小常量 二者差小于它认为相等
    {
        ld mid=(l+r)/2.0;
```

```
            if(check(mid))
                l=mid;
            else
                r=mid;
        }//也可指定循环次数 做个500次即可
        vector<int> ans;
        int t=n;
        while(t)//类似链表形式输出节点
        {
            ans.push_back(t);
            t=pre[t];
        }
        reverse(ans.begin(),ans.end());
        for(int i=0;i<ans.size();i++)
        {
            cout<<ans[i]<<" ";
        }
        cout<<'\n';



    }
    signed main()
    {
        ios::sync_with_stdio(false);
        cin.tie(nullptr);
        cout.tie(nullptr);
        int t=1;
        while(t--)
            solve();
        return 0;
    }
```

T Turtle and Three Sequences CodeForces 2003F

思路

给定a,b,c三个数组，要求取m个角标，使得a不递减，b互不相同，此时c的和的最大值。（1<=n<=3000, 1<=m<=5） 如果直接暴力就是n的m次方，在比较坏的情况下会TLE。这里考虑采用dp[i][j], i用来表示b的不同值状况，j用来表示当前加入的a的最大值，方便后续增加，而dp[i][j]就表示当前情况下c的和的最大值。状压，用二进制位来表示b的不同值状况，最坏需要2的n次方位来表示，这显然是不现实的，所以需要运用到一个随机涂色的技巧（m的值非常小）。 对于正确的一组取值涂上不同颜色的概率是m！/（m^m），当m=5时概率为0.0384，概率是非常小的，所以我们需要不断进行涂色。当T=300时，1 - (1 - 0.0384)^300 = 0.9999，近似于百分百。因此我们选择将不同b值随机上色（0到m-1），执行一百次。 在每次上色中，我们都需要利用滚动数组，对每组i，倒序遍历整个数组，查询当前a[i]可以插入的最大c值方案。这里需要用树状数组维护一下c的最大值，把复杂度降到logn。具体的实现在代码里。

代码

```cpp
#include <bits/stdc++.h>
using namespace std;

#define int long long

const int MAXN = 3e4 + 5;

mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count()); //
随机数模板
template <class T>
inline T rnd(T l, T r)
{
    uniform_int_distribution<T> dist(l, r);
    return dist(rng);
}

int lowbit(int x) // 树状数组lowbit
{
    return (x & -x);
}

vector<vector<int>> dp(32, vector<int>(MAXN, 0));
int n; // dp[i][j]，状压dp，i表示第i个树状数组，其中第i位为1表示该颜色被用了，j记录当
前最后一个a数组数据的大小，方便后续保证有序
       // dp[i][j]表示此时c的和，取最大，用树状数组维护最大值

void update(int x, int pos, int val) // 更新第x个树状数组pos到n的最大前缀
{
    for (int i = pos; i <= n; i += lowbit(i))
    {
        dp[x][i] = max(dp[x][i], val);
    }
}

int query(int x, int pos) // 查询1到pos里的最大值
{
    int res = 0;
    for (int i = pos; i > 0; i -= lowbit(i))
    {
        res = max(res, dp[x][i]);
    }
    return res;
}

signed main()
{
    ios::sync_with_stdio(0);
    cin.tie(0), cout.tie(0);

    int m;
    cin >> n >> m;

    vector<int> a(n + 1), b(n + 1), c(n + 1);
```

```
    for (int i = 1; i <= n; i++)
    {
        cin >> a[i];
    }

    for (int i = 1; i <= n; i++)
    {
        cin >> b[i];
    }

    for (int i = 1; i <= n; i++)
    {
        cin >> c[i];
    }

    int total_sum = (1 << m) - 1; // 总的树状数组个数

    int ans = 0;

    vector<int> color_b(n + 1);

    int T = 300;                 // m=5最好执行300次
    for (int t = 0; t < T; t++) // 对于一组真正的最优解，被染成不同颜色的概率为
m/pow(m, m)
    {
        for (int i = 1; i <= n; i++) // 根据b值随机填充0到m-1种颜色
        {
            color_b[i] = rnd<int>(0, m - 1);
        }

        for (int i = 0; i <= total_sum; i++) // b值重新分配颜色后重置dp
        {
            fill(dp[i].begin(), dp[i].end(), 0);
        }

        for (int i = 1; i <= n; i++)
        {
            int bit = (1 << color_b[b[i]]); // 表示第i位数字被填充

            update(bit, a[i], c[i]); // 初始化

            for (int j = total_sum; j >= 0; j--) // 倒序滚动
            {
                if (j & bit) // 说明bit在当前树被填充过了
                {
                    continue;
                }

                int prev = query(j, a[i]); // 只查询小于等于当前a[i]的方案
                if (prev == 0)             // 说明没有找到
                {
                    continue;
                }
```

```
                int next_j = j | bit;
                update(next_j, a[i], prev + c[i]); // 更新这个数组涂色后的下个
数组
            }
        }

        ans = max(ans, query(total_sum, n)); // 查询本次涂色得到的最后一个树状数
组所保存的最大值
    }

    if (ans == 0) // 如果还是为0说明不存在
    {
        cout << "-1" << "\n";
    }
    else
    {
        cout << ans << "\n";
    }

    return 0;
}
```